

Text Classification

Aijun An

Department of Electrical Engineering and Computer Science

York University

Outline

- ▶ Introduction and applications
- ▶ Traditional methods
 - ▶ Text Representation
 - ▶ Text Preprocessing Steps
- ▶ Advanced techniques for text representation (neural embeddings)

Text Mining

- ▶ It refers to data mining using text documents as data.
 - ▶ A text document could be an article, a web page, a product review, an xml file, an email message, a blog and so on.
- ▶ Tasks of text mining
 - ▶ Text classification
 - ▶ Text clustering
 - ▶ Text summarization
 - ▶ Topic detection
 - ▶ ...

Text Classification

- ▶ Learn a classification model from a set of pre-classified documents
- ▶ Classify new text documents using the learned model
- ▶ Applications
 - ▶ Classify articles into categories
 - ▶ Classify web pages into different categories
 - ▶ Classify emails into different categories
 - ▶ Spam email filtering
 - ▶ ...

Example Applications

- ▶ News topic classification (e.g., Google News)
 $C = \{\text{politics, sports, business, health, tech, ...}\}$
- ▶ Sentiment classification
 $C = \{\text{positive review, negative review, neutral review}\}$
- ▶ “SafeSearch” filtering
 $C = \{\text{pornography, not pornography}\}$
- ▶ Language classification
 $C = \{\text{English, Spanish, Chinese, ...}\}$
- ▶ Email sorting
 $C = \{\text{spam, meeting reminders, invitations, ...}\}$ – user-defined!

Text Representation

- ▶ Most classification learning programs require the examples to be represented as a tuple, which is a vector of attribute values.
- ▶ How to represent a document using a vector of attribute values?
- ▶ Text Representation methods
 - ▶ Traditional methods
 - ▶ Bag of words
 - ▶ Neural-network based embedding methods

Traditional Methods

▶ Attributes

- ▶ “Bag of words” method: Use a set of words as attributes

▶ Attribute values

- ▶ Method 1: use 0 or 1 as attribute value to indicate whether the word appears in the document.
- ▶ Method 2: use the absolute or relative frequency of each word in the document as the attribute value.
- ▶ Method 3: assign a weight to a word in a document using TF-IDF and use the weight as the attribute value

Text Representation (*Cont'd*)

Training data sets:

► Method 1:

	word₁	word₂	...	word_m	Class
document₁	0	1	...	1	C1
document₂	1	0	...	1	C2
...	...	...	...	...	...
document_n	1	0		0	C2

► Method 2 with absolute term frequency:

	word₁	word₂	...	word_m	Class
document₁	0	3	...	1	C1
document₂	2	0	...	3	C2
...	...	...	...	...	...
document_n	5	0		0	C2

Method 3: TF-IDF Term Weighting

▶ TF: term frequency

▶ Definition: $TF = t_{ij}$

▶ frequency of term i in document j

▶ Purpose: makes the frequent words *for the document* more important

▶ IDF: inverted document frequency

▶ Definition: $IDF = \log(N/n_i)$

▶ n_i : number of documents containing term i

▶ N : total number of documents

▶ Purpose: makes rare words *across documents* more important

▶ TF-IDF value of a term i in document j

▶ Definition: $TF \times IDF = t_{ij} \times \log(N/n_i)$

Example: TF-IDF Weighted Vectors

Assume there are three documents in the training set:

Document D1: “yes we got no bananas”

Document D2: “what you got”

Document D3: “yes I like what you got”

	yes	we	got	no	bananas	what	you	I	like
D1:	.18	0.48	0	0.48	0.48	0	0	0	0
D2:	0	0	0	0	0	0.18	0.18	0	0
D3:	0.18	0	0	0	0	0.18	0.18	0.48	.48

Other versions of TF-IDF

- ▶ **Using Relative Document Frequency:**

$$W_{td} = (tf_{td} / |d|) \times \log(N/df_i)$$

where $|d|$ is the length of the document d (i.e., the number of terms the document contains)

- ▶ **Logarithmically Scaled Frequency:**

$$W_{t,d} = \log(1 + tf_{t,d}) \times \log(N/df_t)$$

- ▶ 1 is added in the first term to prevent $\log(0)$.

- ▶ Both prevent bias towards longer documents:

Text Processing for Selecting the Bag of Words (BOW)

- ▶ Word (token) extraction
 - ▶ Extract all the words in a document
 - ▶ Convert them into lower cases
- ▶ Stop words removal
- ▶ Stemming or Lemmatization
- ▶ Selecting words

Stop Words

- ▶ Many of the most frequently used words in English are worthless in text mining – these words are called *stop words*.
- ▶ Examples of stop words
the, of, and, to, a, ...
- ▶ Typically about 400 to 500 such words
- ▶ For an application, there may be additional domain-specific stop words
- ▶ These stop words are often removed from the set of words when representing a document with BOW

Stemming

- ▶ A technique used to find the root/stem of a word.

- ▶ For example:

- ▶ discussed
- ▶ discusses
- ▶ discussing
- ▶ discuss

Stem: discuss

- ▶ Usefulness

- ▶ Reduce the number of words
- ▶ Improve effectiveness of text classification

Example Stemming Rules

- ▶ Remove ending
 - ▶ If a word ends with *s*, preceded by a consonant other than an *s*, then delete the *s*.
 - ▶ If a word ends with *ed*, preceded by a consonant, delete the *ed* unless this leaves only a single letter.
- ▶ Transform words
 - ▶ If a word ends with “ies” but not “eies” or “aies”, then “ies” is replaced with “y”.

Stemming Algorithms

- ▶ Porter stemming algorithm
 - ▶ The most widely used stemming algorithm
 - ▶ Developed by Martin Porter at the University of Cambridge in 1980
 - ▶ <http://www.tartarus.org/~martin/PorterStemmer/> contains source codes in a few languages
- ▶ Other stemming algorithms
 - ▶ Lancaster Stemmer
 - ▶ more aggressive than the Porter Stemmer. It can sometimes over-stem
 - ▶ Snowball Stemmer:
 - ▶ an improvement of the Porter Stemmer

Lemmatization

- ▶ Similar to Stemming
 - ▶ Reducing words to their base or root form,
- ▶ Different from Stemming
 - ▶ Takes into account the vocabulary and morphological analysis of words to remove inflectional endings only
 - ▶ Return **the base or dictionary form** of a word, known as “**lemma**”.
 - ▶ Thus, a lemma is a valid word in the vocabulary, while stemming may result in a stem that is not a valid word
- ▶ More sophisticated than stemming
 - ▶ requires a comprehensive vocabulary and morphological analysis to accurately determine the lemma

Lemmatization

► Examples:

1. Running:
 1. Stemming (e.g., using Porter stemmer): "runn"
 2. Lemmatization: "run"
2. Better:
 1. Stemming: "better"
 2. Lemmatization: "good"
3. Mice:
 1. Stemming: "mice"
 2. Lemmatization: "mouse"
4. Feet:
 1. Stemming: "feet"
 2. Lemmatization: "foot"
5. Flying:
 1. Stemming: "fly"
 2. Lemmatization: "fly"
6. Quickly:
 1. Stemming: "quickli"
 2. Lemmatization: "quickly"

Text Processing for Selecting the Bag of Words

- ▶ Word (token) extraction
 - ▶ Extract all the words in a document
 - ▶ Convert them into lower cases
- ▶ Stop words removal
- ▶ Stemming or lemmatization
- ▶ *Selecting words*

Feature Selection

- ▶ Selecting the “bag of words” to represent documents
- ▶ Why do we need to select?
 - ▶ The number of unique words in a set of documents can be too many.
 - ▶ Learning program may not be able to handle all possible features
 - ▶ Good features can result in higher accuracy

What are Good and Bad Features?

- ▶ Good features: (should be kept)
 - ▶ Co-occur with a particular category
 - ▶ Do not co-occur with other categories
- ▶ Bad features: (best to remove)
 - ▶ Uniform across all categories
 - ▶ Very infrequent (appear 1 or 2 times in the whole training set of documents)
 - ▶ unlikely to be met again
 - ▶ can be noise
 - ▶ co-occurrence with a class can be due to chance

Feature Selection Methods

- ▶ Class independent methods (Unsupervised)
 - ▶ Document Frequency (DF)
 - ▶ Term Strength (TS)
- ▶ Class-dependent methods (Supervised)
 - ▶ Information Gain (IG)
 - ▶ Mutual Information (MI)
 - ▶ χ^2 statistic (CHI)

Document Frequency (DF)

- ▶ Document frequency of a word w :

$DF(w)$ = number of documents containing w

- ▶ Rank the words according to their document frequency
- ▶ Select the first m words with high DF values

Document Frequency (*Cont'd*)

- ▶ Advantages

- ▶ Easy to compute
- ▶ Can remove rare words (hence noise)

- ▶ Disadvantages

- ▶ **Class independent:**

- ▶ If the word appears frequently in many classes, it cannot distinguish the classes well
- ▶ Some infrequent terms can be good discriminators, which cannot be selected by this method.

Information Gain/Mutual Information

- ▶ A measure of importance of the feature for predicting the classes of documents
- ▶ Defined as:
 - ▶ The number of “bits of information” gained by knowing the word is present or absent

$$\begin{aligned} \text{Gain}(w) = & -\sum_{i=1}^k P(C_i) \log P(C_i) \\ & + P(w) \sum_{i=1}^k P(C_i | w) \log P(C_i | w) + P(\bar{w}) \sum_{i=1}^k P(C_i | \bar{w}) \log P(C_i | \bar{w}) \end{aligned}$$

where w is a word and $C_1, C_2, \dots, C_k$ are classes.

- ▶ Rank the words according to their information gain value
- ▶ Select the first m words with high gain values

Information Gain (*Cont'd*)

- ▶ Advantage:
 - ▶ Consider the classes
- ▶ Disadvantage:
 - ▶ Computationally expensive (compared to using DF)
 - ▶ Noisy words occurring only once in the document collection have high IG
- ▶ Solution
 - ▶ Remove rare words (appears 1 or 2 times) first. This can
 - ▶ reduce the amount of computation, and
 - ▶ remove noisy words that have by-chance correlations with the classes.

What Do People Do In Practice with the Bag of Words Representation Method?

- ▶ Rare term removal
 - ▶ rare across the whole collection (i.e. DF is very low)
 - ▶ met in a single document
- ▶ Most frequent term removal (i.e. removing stop words) (*often*)
- ▶ Stemming. (*often*)
- ▶ Use a class-dependent method (e.g., the information gain, chi-square method) to select features.
- ▶ After the BOW presentation, any classification method can be used to learn a classification model.

Limitations of Bag of Words

- ▶ Cannot model phrases
 - ▶ “New York” is split into “New” and “York”
- ▶ Does not consider semantic relationships between words
 - ▶ “happy” and “delighted” have similar meanings
 - ▶ “old bike” vs “used bike”
- ▶ Does not consider the position or order of words in a document
 - ▶ “this is interesting” vs “is this interesting”
- ▶ Does not consider the context a word is in
 - ▶ “Bank of Montreal” vs “river bank”

Beyond Bag of Words

- ▶ N-gram document representation
 - ▶ Bigram
 - a pair of consecutive words in the document
 - ▶ N-gram
 - a sequence of n consecutive words in the document
 - ▶ Document representation
 - Use a set of bi/n-grams to represent documents, or
 - Add them to the bag of words
 - ▶ **Pros:** can represent phrases
 - ▶ **Cons:** WAY too many features to deal with, and many are very sparse.
- ▶ Document representation using *neural embeddings*

Outline

- ▶ Introduction and applications
- ▶ Traditional methods
 - ▶ Text Representation
 - ▶ Text Preprocessing Steps
- ▶ *Advanced techniques for text representation*
 - ▶ *Neural embeddings*
 - ▶ *Text classification with embeddings*

Word Embedding

- ▶ A type of word representation using vectors
 - ▶ Embed words into a vector space
 - ▶ The vector of a word is called its embedding

A 4-dimensional embedding

cat =>	1.2	-0.1	4.3	3.2
mat =>	0.4	2.5	-0.9	0.5
on =>	2.1	0.3	0.1	0.4
...				

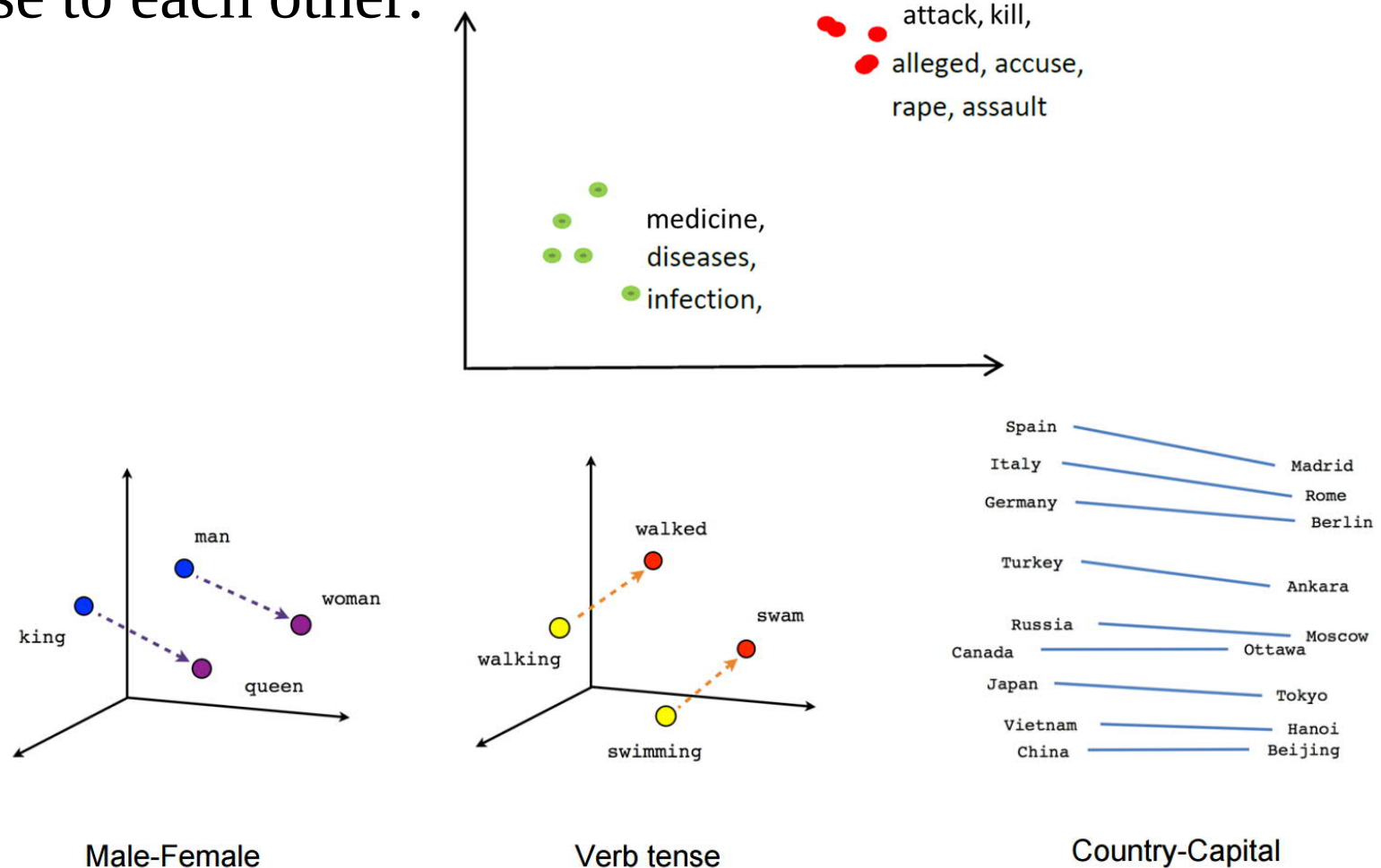
- ▶ Given a vocabulary of words
 - ▶ The embedding of the i^{th} word in the vocabulary is a *d -dimensional vector*:

$$w_i \in \mathbb{R}^d$$

where d is typically in the range 50 to 1000.

Important Feature of Word Embedding

- ▶ In an embedding space, **semantically similar** words are close to each other:



Benefits of Word Embedding

- ▶ Word embeddings is better than one-hot encoding
- ▶ One-hot encoding:
 - ▶ Large dimension: size of vocabulary (could be tens of thousands)
 - ▶ Sparse
 - ▶ Words are considered independent (no semantics is encoded)

```
Rome = [1, 0, 0, 0, 0, 0, ..., 0]
Paris = [0, 1, 0, 0, 0, 0, ..., 0]
Italy = [0, 0, 1, 0, 0, 0, ..., 0]
France = [0, 0, 0, 1, 0, 0, ..., 0]
...
```

- ▶ Word embedding

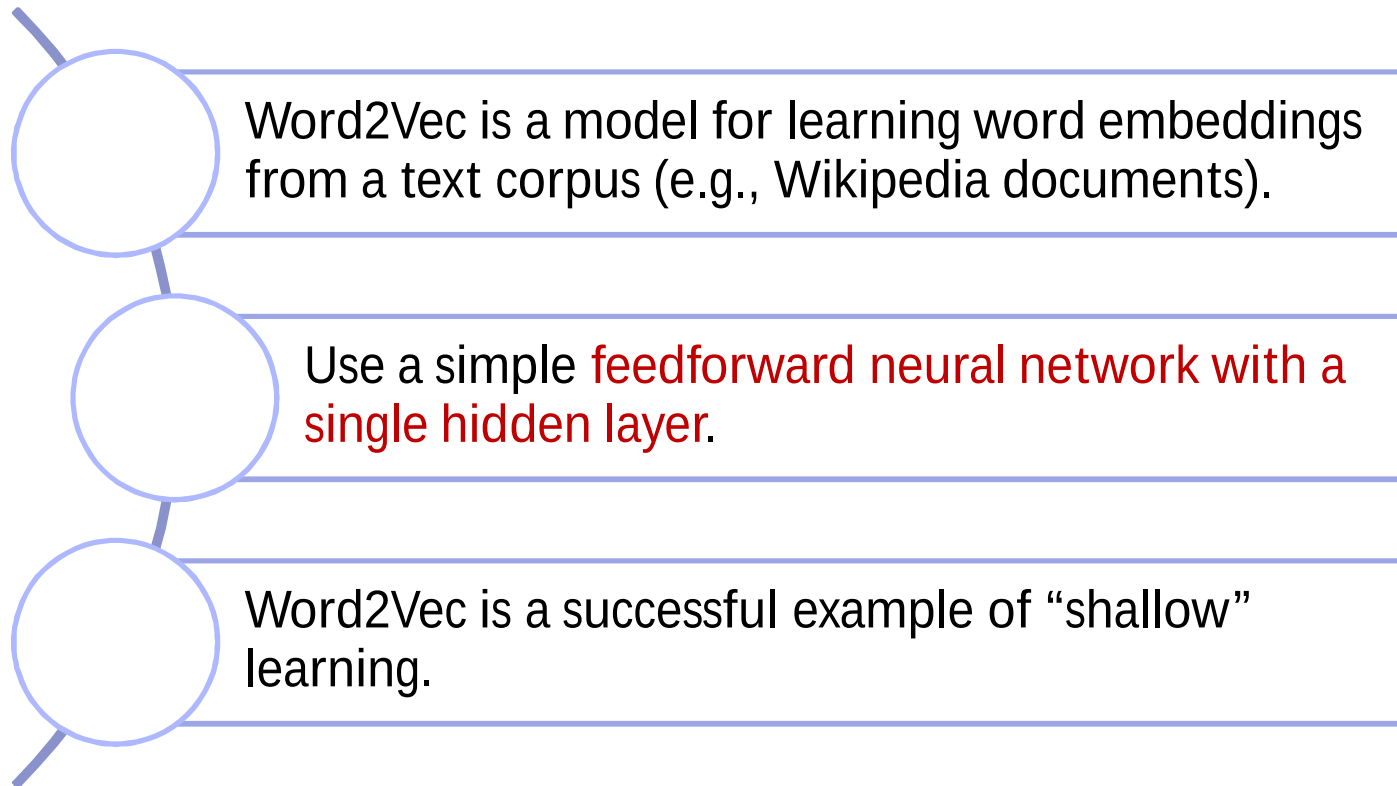
- ▶ Low dimensionality: typically 50 – 1000
- ▶ Dense and distributed
- ▶ Semantically related words are close to each other.

```
Rome = [1.73  0.73 -0.90 -0.62 0.12 -1.35 ...]
Paris = [0.77  1.18 -1.12 -0.75 -0.60 -1.05 ...]
Italy = [1.77 -0.78 -0.95 0.33 0.04 -0.09 ...]
France = [0.67  0.30 -1.05 -0.70 -0.78 0.00 ...]
```

Main Methods for Word Embedding

- ▶ Latent Semantic Indexing (Deerwester et al., '88).
- ▶ Neural Net Language Models (NN-LMs) (Bengio et al., '06)
- ▶ Convolutional Nets for tagging (SENNA) (Collobert & Weston, '08).
- ▶ Supervised Semantic Indexing (Bai et al, '09).
- ▶ Wsabee (Weston et al., '10).
- ▶ Recurrent NN-LMs (Mikolov et al., '10).
- ▶ Recursive NNs (Socher et al., '11).
- ▶ **Word2Vec** (Mikolov et al., '13).
- ▶ GloVe (Pennington et al., '14)
- ▶ BERT (Devlin et al., '18)
- ▶ ROBERTa (Meta AI' 19)
- ▶

Characteristics of Word2Vec



Two Learning Architectures of Word2Vec

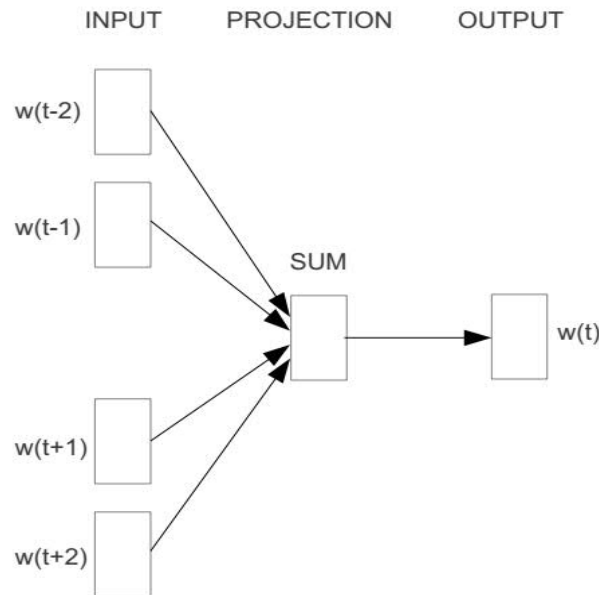
- ▶ Continuous Bag-of-Words (CBOW)
 - ▶ Model is trained to predict **target** words from **context** words
- ▶ Skip-gram
 - ▶ Model is trained to predict **context** words from **target** words

Snapshot of the **training text corpus** (for *context window size* = 2):

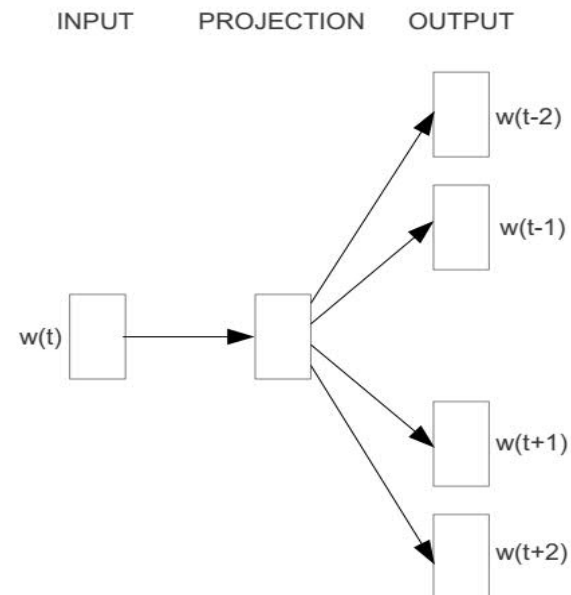
... The **cute** **cat** **jumps** **over** **the** lazy dog ...

$w(t-2)$ $w(t-1)$ $w(t)$ $w(t+1)$ $w(t+2)$
context words target word context words

Word2Vec: Two Learning Architectures



CBOW



Skip-gram

Snapshot of the training text corpus (for context window size = 2):

... The cute cat jumps over the lazy dog ...

$w(t-2)$ $w(t-1)$ $w(t)$ $w(t+1)$ $w(t+2)$
context words target word context words

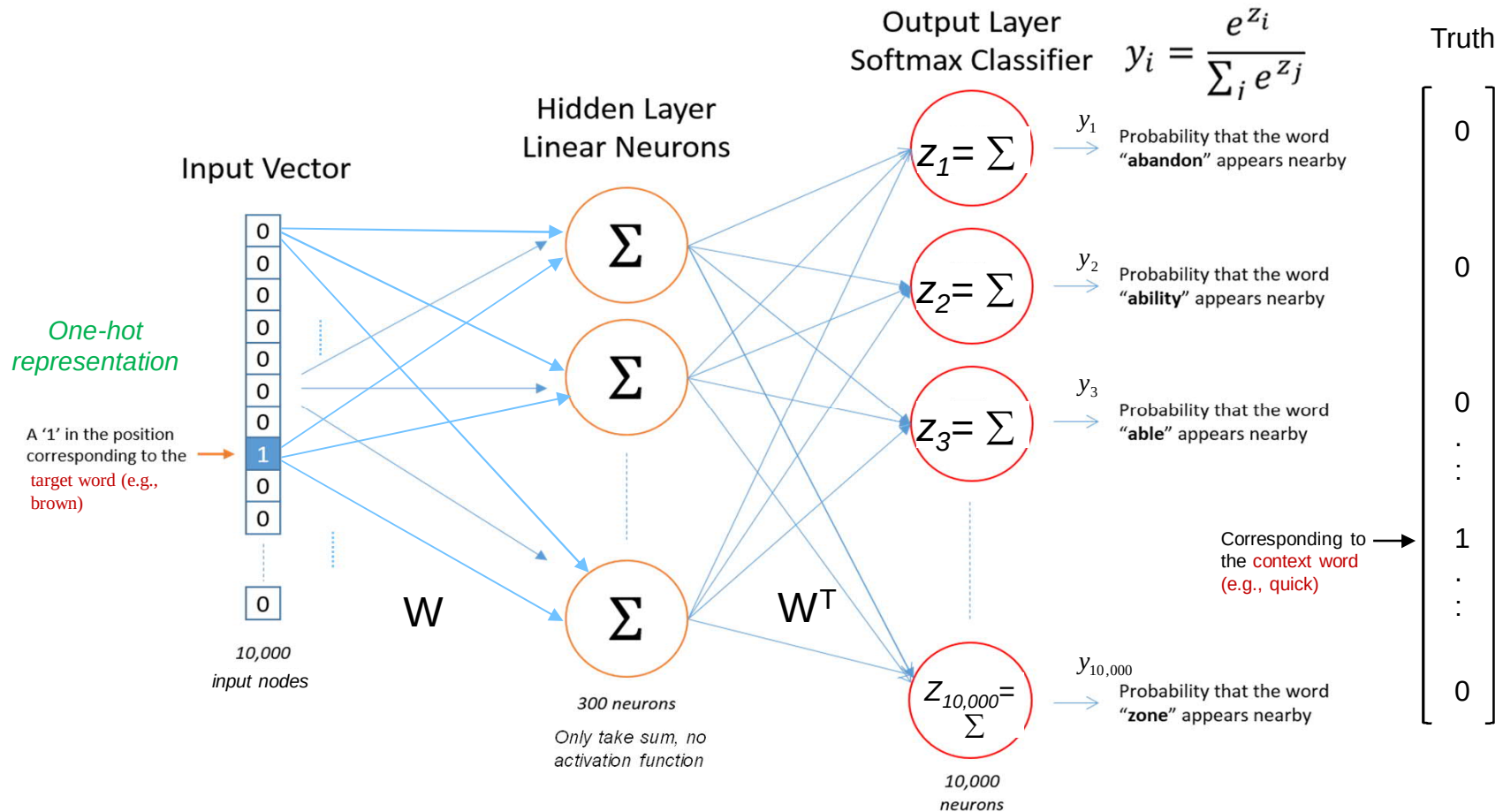
Skip-gram

- ▶ Learn to predict the context words from a target word based on a training corpus
 - ▶ Target word moves from the beginning to the end of the corpus
 - ▶ Context words are the k words before or after the target word (k is the context window size)
 - ▶ Training data contains (target word, context word) pairs:

Source Text	Training Samples						
<table><tr><td>The</td><td>quick</td><td>brown</td></tr></table> fox jumps over the lazy dog. ➡	The	quick	brown	(the, quick) (the, brown)			
The	quick	brown					
<table><tr><td>The</td><td>quick</td><td>brown</td><td>fox</td></tr></table> jumps over the lazy dog. ➡	The	quick	brown	fox	(quick, the) (quick, brown) (quick, fox)		
The	quick	brown	fox				
<table><tr><td>The</td><td>quick</td><td>brown</td><td>fox</td><td>jumps</td></tr></table> over the lazy dog. ➡	The	quick	brown	fox	jumps	(brown, the) (brown, quick) (brown, fox) (brown, jumps)	
The	quick	brown	fox	jumps			
<table><tr><td>The</td><td>quick</td><td>brown</td><td>fox</td><td>jumps</td><td>over</td></tr></table> the lazy dog. ➡	The	quick	brown	fox	jumps	over	(fox, quick) (fox, brown) (fox, jumps) (fox, over)
The	quick	brown	fox	jumps	over		

Skip-gram Model

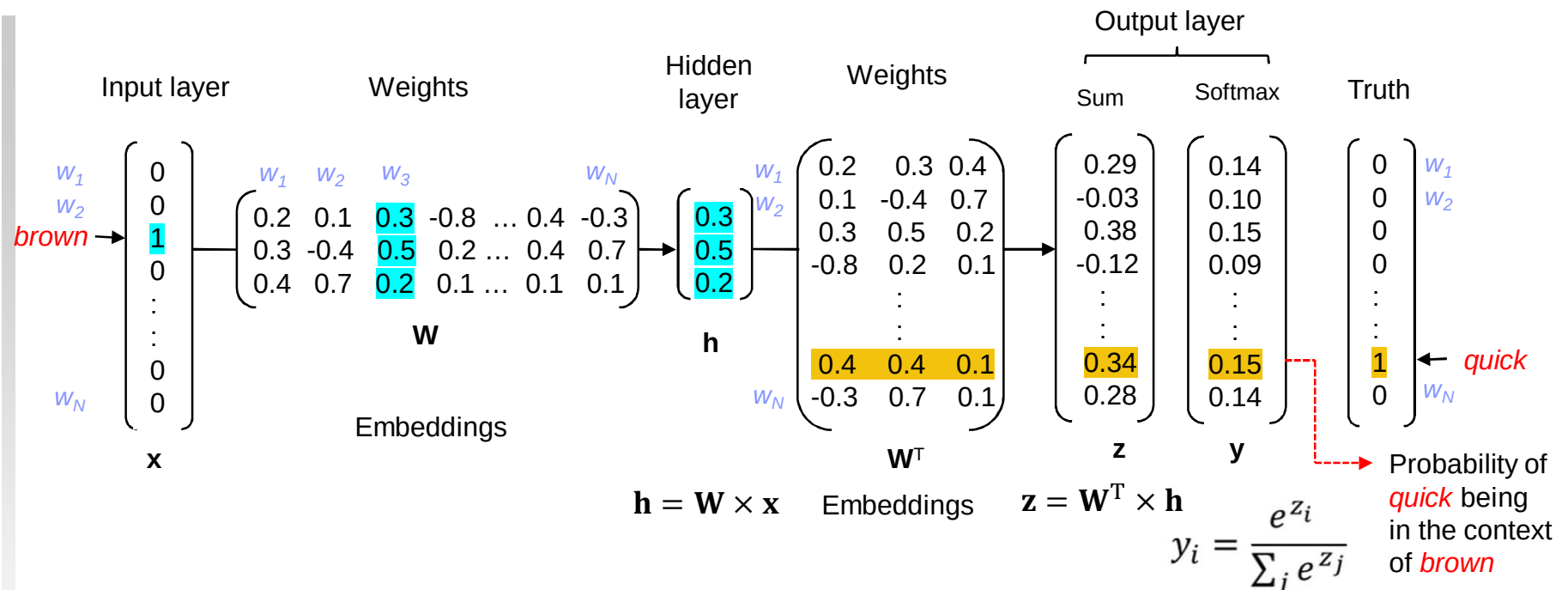
Assume there are 10,000 words in the vocabulary:



- After training, the weights are the word embeddings
- Dimension of the word vector is the number of neurons in the hidden layer

Skip-gram Model Example

Assume training example is (*brown*, *quick*) and embedding dimension is 3:

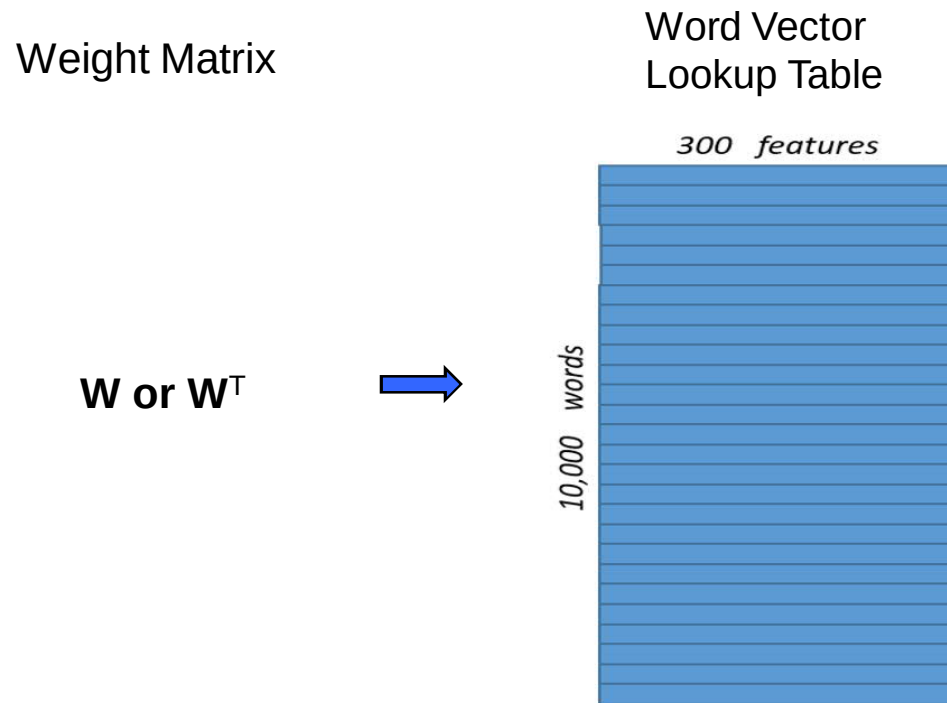


w_i represents i th word in the vocabulary

- z_i is the dot product of the target and context word embeddings, representing the similarity between the two words
- Such training tries to make words close to each other in the text corpus have similar embeddings

Word Vectors: weights

Assume there are 10,000 words in the vocabulary and 300 hidden neurons:



Dimension of the word vector is the number of neurons in the hidden layer

Skip-gram Model with Negative Sampling

- ▶ Since # of output nodes is huge, computation cost is high
- ▶ Negative sampling
 - ▶ Randomly select a small set of non-context words as “negative examples”
 - ▶ Training data contains:
 - ▶ Positive examples:
 - ▶ (target word, context word) pairs
 - ▶ Negative examples:
 - ▶ (target word, random non-context word) pairs

Skip-gram Model with Negative Sampling

▶ Example

The quick brown fox jumps over the lazy dog.

▶ Positive examples for target word “brown”:

(brown, the)

(brown, quick)

(brown, fox)

(brown, jumps)

▶ Negative examples might be:

(brown, by)

(brown, target)

(brown, forever)

(brown, well)

.....

Skip-gram Model with Negative Sampling

- ▶ Use Sigmoid as the activation function at the output layer
 - ▶ Each node is trained to output the probability of the corresponding word in the context of the target word.
- ▶ Give a positive example and a few negative examples (5-20) with the same target word

Positive: (t, c)

Negative: (t, r₁), (t, r₂), ..., (t, r_k)

- ▶ Only the output nodes corresponding to the context word c and random words r₁, r₂, ... r_k are activated and used in backpropagation training
- ▶ Loss function:

$$L(\theta) = -P(\mathbf{c}|\mathbf{t}) + \frac{1}{k} \sum_{i=1}^k P(\mathbf{r}_i|\mathbf{t})$$

$$P(\mathbf{c}|\mathbf{t}) = \text{sigmoid}(\mathbf{c} \cdot \mathbf{t})$$

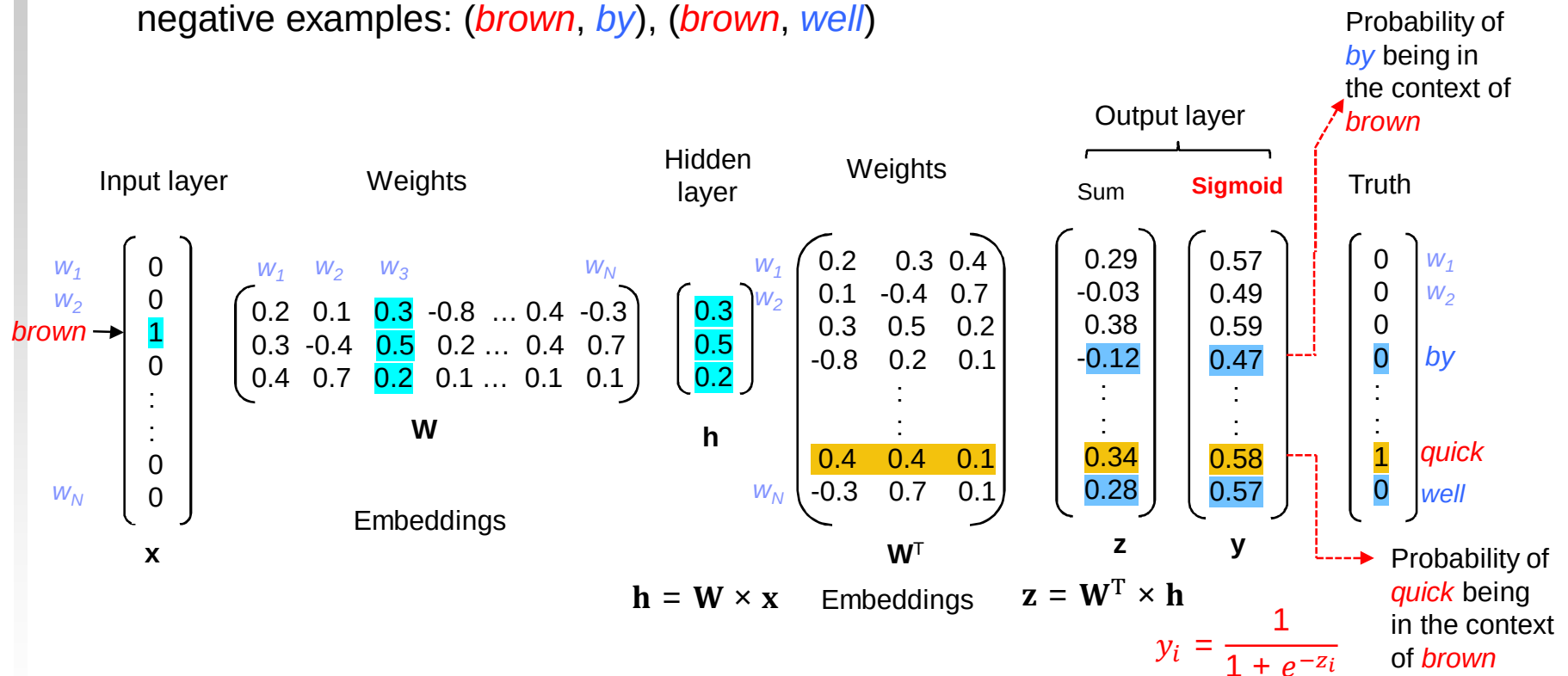
$$P(\mathbf{r}_i|\mathbf{t}) = \text{sigmoid}(\mathbf{r}_i \cdot \mathbf{t})$$

Skip-gram Model with Negative Sampling

Assume:

positive example: (*brown*, *quick*)

negative examples: (*brown*, *by*), (*brown*, *well*)



$$L(\theta) = -P(\mathbf{c}|\mathbf{t}) + \frac{1}{k} \sum_{i=1}^k P(\mathbf{r}_i|\mathbf{t}) = -0.58 + 0.47 + 0.57$$

Some Other Word Embedding Models

- ▶ **GloVe**

- ▶ does not rely just on local statistics (local context information of words)
- ▶ incorporates global statistics (word co-occurrence)

- ▶ **FastText**

- ▶ consider subword information, making it effective for understanding morphologically rich languages or misspelled words.

- ▶ **ELMo**

- ▶ Context-dependent

Some Other Word Embedding Models

- ▶ BERT (Bidirectional Encoder Representations from Transformers)
 - ▶ Based on transformers
 - ▶ Context-dependent
 - ▶ “financial bank” vs “river bank”
 - ▶ Trained with unlabeled text on two NLP tasks:
 - ▶ Masked language model
 - ▶ Next sentence prediction
 - ▶ Two models
 - ▶ BERT Base: 12 layers and 110 million parameters
 - ▶ BERT Large: 24 layers and 340 million parameters
- ▶ RoBERTa, ALBERT, DistilBERT, etc.
 - ▶ Variations and optimizations of BERT that aim to improve aspects like training speed, memory efficiency, and accuracy.

Some Other Word Embedding Models

- ▶ GPT (Generative Pretrained Transformer)
 - ▶ primarily designed for generation tasks
 - ▶ can be adapted for classification by using the representations from the last layer.